FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 5

Le cœur de l'application — les incidents

Durée : 5 jours Form Request Règles métier Dates Événements Eloquent

Intentions pédagogiques

C'est la semaine la plus dense. L'enjeu n'est pas seulement de faire marcher le formulaire mais de **placer chaque règle au bon endroit** (validation, contrôleur, modèle) et de comprendre **pourquoi**.

Faites-lui tenter l'attaque par mass assignment lui-même (ajouter un champ caché via les outils du navigateur) : c'est la meilleure façon de retenir la leçon.

Déroulé conseillé

Jour	Activité
J1	`IncidentRequest` complet. Création d'un incident (formulaire + `store`).
J2	Page de détail. Méthodes `dureeMinutes()` et `dureeLisible()`.
J3	Modification, règles de résolution (dates automatiques, `required_if`).
J4	Tentative de mass assignment, impression, exercices SQL.
J5	Tests manuels de toutes les règles, point hebdomadaire.

Corrigé : fichiers de référence

- `app/Http/Requests/IncidentRequest.php`
- `app/Http/Controllers/IncidentController.php` (méthodes `store`, `update`, `avecDateResolution`)
- `app/Models/Incident.php` (`booted`, `dureeMinutes`, `dureeLisible`, `formaterDuree`)
- `resources/views/incidents/form.blade.php`, `show.blade.php`

Gestion de la date de rétablissement :

```
private function avecDateResolution(array $donnees): array
{
    $termine = StatutIncident::from($donnees['statut'])->estTermine();

    if ($termine && empty($donnees['resolu_at'])) {
        $donnees['resolu_at'] = now();
    }
    if (! $termine) {
        $donnees['resolu_at'] = null;
    }

    return $donnees;
}
```

Formatage de la durée :

```
public static function formaterDuree(int|float|null $minutes): string
{
    if ($minutes === null) {
        return '–';
    }
    $minutes = (int) round($minutes);
    $jours = intdiv($minutes, 1440);
    $heures = intdiv($minutes % 1440, 60);
    $reste = $minutes % 60;

    return trim(($jours ? "{$jours}j " : '').($heures ? "{$heures}h " : '')."{$reste}min");
}
```

Corrigé des exercices SQL

```sql
-- 1. Incidents actifs, du plus ancien au plus récent
SELECT reference, titre, statut, debut_at
FROM incidents
WHERE statut IN ('ouvert', 'en_cours')
ORDER BY debut_at;

-- 2. Durée des incidents résolus
SELECT reference, TIMESTAMPDIFF(MINUTE, debut_at, resolu_at) AS duree_min
FROM incidents
WHERE resolu_at IS NOT NULL;

-- 3. Résolus sans cause racine
SELECT reference FROM incidents
WHERE statut IN ('resolu', 'clos') AND (cause_racine IS NULL OR cause_racine = '');
-- Aucun : la Form Request l'interdit. MAIS un INSERT SQL direct le pourrait :
-- la règle vit dans l'application, pas dans la base. Bon sujet de discussion.

-- 4. Déclarant et assigné
SELECT i.reference, d.name AS declare_par, a.name AS assigne_a
FROM incidents i
JOIN users d ON d.id = i.declare_par_id
LEFT JOIN users a ON a.id = i.assigne_a_id;
-- Avec un JOIN simple sur l'assigné, les incidents non assignés disparaissent.
```

Erreurs fréquentes

Erreur	Symptôme	Piste
`$request->all()` dans `create`	Faille de mass assignment	`$request->validated()` + champs serveur
`declare_par_id` en champ caché	Usurpation possible	Le fixer côté serveur
Format `datetime-local`	Champ vide en modification	`format('Y-m-d\TH:i')`
`$incident->debut_at->format()` sur un nouvel incident	« Call to a member function on null »	Opérateur `?->`
Durée négative ou bizarre	`diffInMinutes` dans le mauvais sens	Depuis `debut_at` vers `resolu_at ?? now()`
Comparaison d'enum avec une chaîne	`$incident->statut === 'resolu'` toujours faux	Comparer à `StatutIncident::Resolu`

Questions pour le point hebdomadaire

1. **Montre-moi ta tentative de mass assignment. Pourquoi a-t-elle échoué ?** *Attendu :* `validated()` ignore les champs non déclarés dans les règles, et `declare_par_id` est forcé côté serveur.

2. **Pourquoi la durée est-elle calculée et non stockée ?** *Attendu :* elle change en permanence tant que l'incident est ouvert ; la stocker créerait une donnée fausse.
3. **Pourquoi la cause racine est-elle obligatoire à la résolution ?** *Attendu :* sans elle, on ne peut pas éviter que l'incident se reproduise (amélioration continue, analyse des récurrences).
4. **Que fait `required_if:statut,resolu,clos` ?** *Attendu :* rend le champ obligatoire uniquement si `statut` vaut `resolu` ou `clos`.
5. **Ta règle « cause racine obligatoire » protège-t-elle aussi les insertions SQL directes ?** *Attendu :* non ; elle n'existe que dans l'application. Pour une garantie en base, il faudrait une contrainte `CHECK`.

Grille d'évaluation (sur 20)

Critère	Points
Création d'incident fonctionnelle, référence générée	3
Form Request complète, messages en français	3
Règles de résolution (cause / solution, dates auto, réouverture)	4
Page de détail complète et lisible	3
Protection mass assignment (démontrée)	2
Durée calculée et bien formatée	2
Exercices SQL (dont le `LEFT JOIN`)	3

Adapter le rythme

- **En avance** : filtrage JavaScript des équipements selon le site, ou ajout d'une contrainte `CHECK` MySQL `resolu_at >= debut_at` dans une migration.
- **En difficulté** : fournir `IncidentRequest` ; il se concentre sur le contrôleur et la page de détail. Les règles 4 et 6 peuvent passer en semaine 6.